

Architecting Seamless Navigation in Vanilla JS RPGs: A Strategy for Hardcoded Bidirectional Linking Using the History API and State Management

Foundational Navigation Control with the History API

The cornerstone of creating a seamless, bidirectional linking system for a pixel RPG built with vanilla HTML, CSS, and JavaScript lies in mastering the browser's History API ^{16 70}. This set of tools allows an application to programmatically manipulate the session history stack, enabling the illusion of navigating between distinct pages without triggering a costly full-page reload ⁹. This capability is fundamental to building Single Page Application (SPA)-like experiences within a traditional multi-section website structure ⁴⁰. The core components of this API are the `pushState()`, `replaceState()`, and `popstate` event, which together provide complete control over forward and backward navigation flows ⁹⁶.

For forward navigation, where a player moves from one game section to another (e.g., from a town to a dungeon), the `history.pushState()` method is essential ²³. Its function is to add a new entry onto the browser's session history stack, thereby making a subsequent click of the back button possible ⁴. When `pushState()` is called, it accepts three parameters: a state object, a title (which is often ignored by modern browsers), and a URL ²⁶. The state object is particularly powerful; it can store a serializable snapshot of the game's context at the moment of navigation ¹⁷. For instance, this object could contain a simple identifier for the destination section, such as `{ sectionId: 'dungeon_1' }` ¹²⁶. This data becomes readily available when the user navigates backward. Critically, `pushState()` updates the browser's address bar with the new URL but does not cause the page to load, allowing for a smooth transition where custom JavaScript logic can render the new game section in place of the old one ¹²⁵. It is important to avoid confusing the user by adding history entries too frequently, as this can lead to a bloated and confusing history trail ⁷. The primary limitation of the `state` object is its reliance on structured cloning, meaning it cannot contain non-serializable

values like functions, undefined, or complex circular references, which would throw a `DataCloneError` ⁸³. Therefore, the state object must be a plain JavaScript object containing only primitive data types like strings, numbers, or nested objects ²⁸.

Intercepting backward navigation is achieved through the `popstate` event, which is fired on the `window` object whenever the active history entry changes ⁶ ¹²⁴. This event is triggered specifically by user actions that alter the history stack, such as clicking the browser's back or forward buttons, or programmatically calling `history.back()` or `history.forward()` ²⁵ ⁸⁸. It is a common misconception that calling `pushState` or `replaceState` directly will trigger a `popstate` event; this is not the case ⁵. The event listener receives a `PopStateEvent` object, which contains a `state` property. This property holds a copy of the state object that was provided to `pushState` when the new history entry was created ⁴. By attaching a listener to this event, the application can capture the identifier of the previously visited section and execute the necessary logic to restore its state and appearance ²⁴. To ensure the application starts in the correct state if a user lands on a deep link (e.g., a direct URL to a specific dungeon), it is a best practice to initialize the view by checking the current `history.state` on `DOMContentLoaded` ¹⁴⁰.

The `history.replaceState()` method serves a different purpose than `pushState()`. Instead of adding a new entry to the history stack, it modifies the current top-most entry ³¹. This is useful for updating the URL to reflect a change within the same section—for example, changing a character's equipment in a menu or filtering a list of items—without cluttering the navigation history with redundant entries ⁸⁵ ⁹⁵. While both methods accept the same parameters (state object, title, and URL), their impact on the history stack is fundamentally different ¹⁹. Since `replaceState` does not create a new history point, it cannot be used to establish a bidirectional link between two distinct game sections. Its utility is limited to transient state modifications within a single view. Understanding the precise difference between these two methods is crucial for maintaining a clean and predictable user navigation experience ³¹. An advanced use case for `replaceState` involves modifying the state object associated with the current history entry in response to a `popstate` event, allowing for dynamic behaviors like showing or hiding modals on back/forward navigation ¹³⁷.

Feature	<code>history.pushState()</code>	<code>history.replaceState()</code>
History Stack	Adds a new entry to the top of the stack ²³ .	Updates the current entry at the top of the stack ³¹ .
Back Button	Creates a new history point that can be navigated back to ⁶² .	Does not create a new point; the back button skips over the replaced state ⁹⁵ .
Primary Use Case	Navigating to a new section or game state ²³ .	Modifying the URL or state within the current section without adding a history step ⁸⁵ .
Trigger for pop state	Triggers a <code>popstate</code> event when the user navigates forward to this new entry ⁸⁸ .	Does not trigger a <code>popstate</code> event ⁵ .

In summary, the History API provides the necessary plumbing for a bidirectional linking system. By strategically combining `pushState` for forward progression and listening for the `popstate` event to handle backward navigation, developers can build a robust and intuitive navigation experience that feels like browsing multiple pages while operating as a single interactive application.

Architecting Game State for Seamless Transitions

Preserving the integrity of the game state during transitions between sections is paramount for a coherent and immersive player experience. A retro-style pixel RPG involves numerous dynamic elements—player health, inventory, quest progress, enemy positions, and NPC dialogue flags—that must persist seamlessly as the player moves through different areas. Hardcoding the navigation links does not preclude the use of sophisticated architectural patterns for state management; in fact, doing so is essential for preventing the codebase from becoming unmanageable and brittle. The sources indicate two powerful, complementary approaches: the State Machine pattern for managing game flow and a centralized state object as the single source of truth for all game data.

The State Machine pattern offers a structured way to model the game's logic by defining a finite set of states and the valid transitions between them ⁷¹. In the context of the RPG, states could be distinct sections like `TOWN`, `FOREST_PATH`, `DUNGEON_1`, or UI windows like `INVENTORY_WINDOW` ⁷². The machine enforces rules about which transitions are permissible, effectively untangling complex conditional logic that might otherwise become tangled in event handlers ⁷³. For example, a state machine could explicitly define that the `INVENTORY_WINDOW` can only be entered from either the `TOWN` or `DUNGEON_1` states. This aligns perfectly with a hardcoded linking system, as the

connections themselves are hard-coded into the machine's transition rules. This pattern helps manage complexity by allowing developers to reason about a small part of the game's behavior in isolation ⁷².

Complementing the State Machine is the concept of a centralized state object, often referred to as the `GameState` object. This pure JavaScript object acts as the "single source of truth" for the entire application ⁴². It is a comprehensive data structure that serializes all relevant aspects of the game's current situation: the player's name, level, and statistics; a list of collected items; key-value pairs tracking completed quests or spoken-to NPCs; and the current coordinates within the world ³³. When transitioning to a new section, the application logic reads from this master object to determine how to render the environment and UI. When a player enters a new area, the `GameState` object is updated with the new context, such as setting `currentState.section = 'DUNGEON_1'` and populating it with the initial data for that dungeon ³³. Crucially, because this state resides in memory and not in the DOM, it persists across section changes ¹⁰¹. When the user navigates backward via the `popstate` event, the application retrieves the previous state information from the history stack's `event.state` object, uses it to reconstruct the `GameState` object, and then re-renders the UI accordingly ²⁴.

This approach stands in stark contrast to trying to store state within the DOM itself, which is volatile and difficult to manage reliably. While client-side storage APIs like `localStorage` and `sessionStorage` exist, they are less suitable for this specific use case. These APIs are designed for persistence across browser sessions, whereas the requirement is for transient state management within a single session ¹¹⁷. Furthermore, they are strictly limited to storing string data, necessitating manual serialization (e.g., using `JSON.stringify`) before saving and deserialization (e.g., using `JSON.parse`) upon retrieval ²⁸. This adds unnecessary overhead and complexity compared to using a native JavaScript object for intra-session state management. If the game state is particularly complex, a custom serialization function may be required to convert objects into a string format that can be saved, followed by a corresponding deserialization function to rehydrate them later ^{27 91}. However, for a hardcoded system focused on session-based continuity, a well-designed central state object remains the most direct and flexible solution.

Optimizing Visual Transitions through Efficient DOM Manipulation

To achieve the "seamless" transitions characteristic of a polished game, raw Document Object Model (DOM) manipulation must be executed with a keen focus on performance. Abruptly removing one section's content and instantly displaying another creates a jarring user experience that undermines the goal of a fluid, retro-style RPG. The provided research highlights several strategies and techniques for creating smooth animations and optimizing DOM operations to ensure high performance, even on less powerful devices. These techniques revolve around leveraging CSS for hardware-accelerated animations, employing efficient JavaScript-based animation loops, and applying best practices for DOM modification.

CSS transitions are consistently identified as a highly performant method for creating smooth visual effects ¹¹. Animating properties like `opacity` and `transform` (e.g., `translateX`, `scale`, `translate3d`) is particularly recommended because these properties can often be offloaded to the GPU, resulting in smoother animations and reduced CPU load ^{12 109}. Using `display: none` to hide an element, while effective at removing it from the document flow, can trigger expensive layout recalculations (reflows) and repaints, especially if it causes surrounding elements to shift ^{29 41}. A superior alternative is to manage visibility through CSS properties. For instance, animating `opacity` from 1 to 0 creates a fade-out effect, while carefully combining this with a transition on `visibility` ensures the element is removed from interaction completely once the fade is complete ¹⁰⁸. Modern CSS features like the View Transition API offer an even more powerful, native solution for defining cross-fade transitions between different parts of the document, providing sophisticated visual continuity with minimal code ^{46 47}. Even without this newer API, manually applying and removing CSS classes to trigger predefined transitions is a standard and effective technique for achieving smooth fades and slides ^{15 51}.

For more complex or custom animations that go beyond simple fades, such as parallax camera effects or intricate sprite movements, JavaScript-based animation loops using `requestAnimationFrame` are the industry standard ¹⁴. Unlike `setTimeout` or `setInterval`, which can lead to janky or inconsistent frame rates, `requestAnimationFrame` schedules a callback to run before the browser's next repaint, synchronizing the animation with the display's refresh cycle to produce smooth 60fps motion ⁶⁴. This method also inherently helps prevent "layout thrashing," a performance killer caused by mixing synchronous DOM reads and writes in a loop ⁶³. By

batching DOM reads (e.g., getting an element's position) first, and then performing all DOM writes (e.g., updating styles) in a single tick, `requestAnimationFrame` ensures the browser can optimize its rendering pipeline efficiently ^{65 66}.

Regardless of the animation method chosen, the underlying DOM manipulation must be performed efficiently. Direct and repeated manipulation of the DOM is notoriously slow ²¹. Best practices dictate caching references to frequently accessed DOM elements to avoid repeated lookups ¹¹². When generating large amounts of new content for a section, it is inefficient to append nodes one by one. Instead, a `DocumentFragment` should be used to build the new DOM tree offline in memory; once the fragment is complete, it can be inserted into the live DOM in a single operation, minimizing reflows and repaints ^{37 113}. To clear the contents of a container before inserting new content, modern and efficient alternatives to clearing `innerHTML` include the `Element.replaceChildren()` method, which is cleaner and avoids the security risks and parsing overhead associated with `innerHTML` ^{34 39}. By combining performant CSS for simple transitions, `requestAnimationFrame` for complex animations, and efficient DOM techniques like `DocumentFragment` and `replaceChildren`, a developer can build a visually rich and responsive pixel RPG that maintains high performance throughout its lifecycle.

Synthesized Implementation Strategy and Considerations

Synthesizing the findings from the analysis of the History API, state management patterns, and DOM optimization techniques yields a robust, hardcoded implementation strategy for a bidirectional linking system in a vanilla JavaScript pixel RPG. This approach prioritizes seamless navigation, persistent game state, and high performance, directly addressing the core requirements of the research goal. The strategy is not merely a collection of disparate techniques but a cohesive architecture where each component plays a critical role in delivering a polished user experience.

The first step is to architect the application around a centralized `GameState` object and a routing mechanism. This involves hardcoding a representation of each game section (e.g., `const SECTIONS = { TOWN: { ... }, DUNGEON_1: { ... } };`). A central `GameManager` class or a simple module would then manage the application's flow. This manager would maintain the current section ID and a history stack of

previously visited sections, acting as the orchestrator for all navigation events. This aligns with the principle of having a single source of truth for the game's state, ensuring all dynamic data is preserved independently of the DOM ³³ ⁴².

Forward navigation is handled by a `goToSection(sectionId)` method within the `GameManager`. This method performs a sequence of actions: first, it calls `history.pushState({ sectionId }, '', /${sectionId})` to create a new history entry and update the URL ²⁶. Immediately after, it initiates a smooth exit animation for the current section's content, likely using a CSS class to trigger a fade-out or slide-out effect ¹⁵. Upon the completion of this exit animation, a callback function is invoked to render the new section. This rendering process involves querying the `SECTIONS` object for the new section's data and efficiently manipulating the DOM, for example, by using `document.getElementById('game-container').replaceChildren(newContentFragment)` to swap the old content in a single, optimized operation ³⁴ ¹¹³. Finally, the `GameManager` updates its internal state to reflect the new location.

Backward navigation is handled automatically by the browser but is managed by the application's logic. An event listener is attached to the `window` object for the `popstate` event ¹²⁴. When the user clicks the browser's back button, this event fires. The event handler extracts the `sectionId` from the `event.state` object, which was passed to `pushState` when the forward navigation occurred ⁴. The application then triggers an entrance animation for the newly revealed section, perhaps fading it in ⁵¹. Once this entrance animation completes, the rendering logic is called again, this time using the retrieved `sectionId` to display the appropriate UI and game world. To ensure the application is always in the correct state on initial load or when navigating to a deep link, a check should be performed on `DOMContentLoaded` to render the view based on the current `history.state` ¹⁴⁰.

Despite its effectiveness, this hardcoded approach has notable limitations. The primary challenge is scalability; as the number of sections and interconnecting paths grows, maintaining a hardcoded map of links and transitions becomes increasingly tedious and error-prone. While this is manageable for smaller projects, it can quickly become a maintenance burden ⁷³ ⁷⁴. Another significant consideration is the complexity of state serialization. While passing a simple section ID in the `pushState` state object is straightforward, preserving the detailed state of a complex section (e.g., the exact position of every enemy in a dungeon) requires a custom serialization and deserialization function, adding a layer of complexity to the hardcoded solution ⁹¹. Finally, developers must be mindful of subtle differences in History API behavior across various browsers,

particularly concerning `popstate` events on initial page loads, which may require specific workarounds to ensure consistent functionality ⁹⁴ ¹³⁸. In conclusion, the proposed strategy provides a solid foundation for building a seamless, bidirectional linking system. It leverages the History API for control, a centralized state object for persistence, and performant DOM techniques for visual polish, offering a powerful and self-contained solution for a vanilla JavaScript pixel RPG.

Reference

1. Simple 2D RPG game created using Vanilla JavaScript · GitHub <https://github.com/adhywiranata/rpg-vanilla-js>
2. Working with the History API - MDN Web Docs https://developer.mozilla.org/en-US/docs/Web/API/History_API/Working_with_the_History_API
3. History API - MDN Web Docs - Mozilla https://developer.mozilla.org/en-US/docs/Web/API/History_API
4. History: `pushState()` method - Web APIs | MDN <https://developer.mozilla.org/en-US/docs/Web/API/History/pushState>
5. How to get notified about changes of the history via `history.pushState`? <https://stackoverflow.com/questions/4570093/how-to-get-notified-about-changes-of-the-history-via-history-pushstate>
6. Window: `popstate` event - Web APIs | MDN https://developer.mozilla.org/en-US/docs/Web/API/Window/popstate_event
7. Must Know JavaScript API — History API | by Chris1993 <https://javascript.plainenglish.io/must-know-javascript-api-history-api-409b3d345ad8>
8. files - en-us - web - api - history - GitHub <https://github.com/mdn/content/blob/main/files/en-us/web/api/history/state/index.md?plain=1>
9. Working with the History API - MDN Web Docs https://mdn2.netlify.app/en-us/docs/web/api/history_api/working_with_the_history_api/
10. Smoothly Reverting CSS Animations <https://pragmaticpineapple.com/smoothly-reverting-css-animations/>
11. CSS and JavaScript animation performance - MDN Web Docs https://developer.mozilla.org/en-US/docs/Web/Performance/Guides/CSS_JavaScript_animation_performance

12. Why moving elements with `translate()` is better than `pos` ... <https://paulirish.com/2012/why-moving-elements-with-translate-is-better-than-posabs-topleft/>
13. Creating Buttery Smooth Animations in CSS <https://www.youtube.com/watch?v=wsysYgOdGDY>
14. `requestAnimationFrame` Guide - 33 JavaScript Concepts <https://33jsconcepts.com/beyond/concepts/requestanimationframe>
15. You can now animate page navigation. With CSS ... https://www.linkedin.com/posts/theosoti_you-can-now-animate-page-navigation-with-activity-7425101029945282560-RsFW
16. Using the History API - JavaScript Tutorial <https://www.youtube.com/watch?v=4GqUAaLgpgs>
17. Hack the browser history - by Richard LIU <https://medium.com/@liuzhenglaichn/hack-the-browser-history-167f73ac403c>
18. Exploring the History `pushState` and `replaceState` Methods <https://www.youtube.com/watch?v=j1ZHuyhHApg>
19. How to Modify the Browser History in Complex HTML5 and ... <https://www.sitepoint.com/javascript-history-pushstate/>
20. Getting Started With The History API - Treehouse Blog <https://blog.teamtreehouse.com/getting-started-with-the-history-api>
21. Best Practices for Speeding Up Your Web Site <https://developer.yahoo.com/performance/rules.html>
22. VanillaJs Routing (with `history.pushState`) | Write Once Don't ... <https://www.youtube.com/watch?v=SLYwSB4SQiw>
23. JavaScript History `pushState` <https://www.javascripttutorial.net/web-apis/javascript-history-pushstate/>
24. simple example of `pushState`/`popState` navigation <https://gist.github.com/9dcd343f84f69cab3485>
25. JavaScript `window.history.pushState` tutorial <https://medium.com/@nagachaitanyakonada/javascript-window-history-pushstate-tutorial-56e2126eaff1>
26. `History.pushState()` - Web APIs <https://udn.realityripple.com/docs/Web/API/History/pushState>
27. Serializing inventory with my custom save system | Opsive <https://www.opsive.com/forum/index.php?threads/serializing-inventory-with-my-custom-save-system.6547/>
28. Serializing JavaScript Objects - null program <https://nullprogram.com/blog/2013/03/11/>
29. Minimising Layout and Layout thrashing for 60 FPS <https://www.harrytheo.com/blog/2021/09/dom-reflow-and-layout-thrashing/>

30. Freecodecamp's post <https://www.facebook.com/freecodecamp/posts/badges-are-popular-but-theyre-often-quite-simple-and-frankly-boring-luckily-some/1068002529145847/>
31. Pushing and Popping with the History API <http://html5doctor.com/history-api/>
32. lrusso/RPGGame: RPG Game in JavaScript <https://github.com/lrusso/RPGGame>
33. Making a turn based RPG game javaScript project example <https://dustinpfister.github.io/2021/12/10/js-javascript-example-turn-based-rpg/>
34. Element: replaceChildren() method - Web APIs | MDN <https://developer.mozilla.org/en-US/docs/Web/API/Element/replaceChildren>
35. Remove all child elements of a DOM node in JavaScript <https://stackoverflow.com/questions/3955229/remove-all-child-elements-of-a-dom-node-in-javascript>
36. Element.insertAdjacentHTML() - Web APIs <https://mdn2.netlify.app/en-us/docs/web/api/element/insertadjacenthtml/>
37. Modifying the document <https://javascript.info/modifying-document>
38. Element: insertAdjacentHTML() method - Web APIs | MDN <https://developer.mozilla.org/en-US/docs/Web/API/Element/insertAdjacentHTML>
39. Modern alternatives to innerHTML and appendChild() <https://marian-caikovski.medium.com/modern-alternatives-to-innerhtml-and-appendchild-296b9e5a5d28>
40. Single-page application routing in vanilla JavaScript using the power ... <https://medium.com/@george.norberg/history-api-getting-started-36bfc82ddefc>
41. Does 'display:none' improve or worsen performance? <https://stackoverflow.com/questions/62709969/does-displaynone-improve-or-worsen-performance>
42. Saving and Loading Data with Web Storage - Grasp <https://paths.grasp.study/modules/b4c3ab00-6660-4162-a18c-d2fdab877c31/lessons/70c8e92a-77b9-4445-8531-7d19e204a7f3>
43. JavaScript Game Development: One Codebase - Many 2D Games <https://www.youtube.com/watch?v=orvi9o88xyY>
44. Cross-Document View Transitions: The Gotchas Nobody Mentions <https://css-tricks.com/cross-document-view-transitions-part-1/>
45. How to: Game UI in Browser Games - YouTube <https://www.youtube.com/watch?v=U62vi6H0lC4>
46. View Transition API: Single Page Apps Without a Framework <https://www.debugbear.com/blog/view-transitions-spa-without-framework>
47. Bringing React's to vanilla JS - Plain Vanilla <https://plainvanillaweb.com/blog/articles/2025-06-12-view-transitions/>

48. View transitions - Astro Docs <https://docs.astro.build/en/guides/view-transitions/>
49. Same-document view transitions for single-page applications <https://developer.chrome.com/docs/web-platform/view-transitions/same-document>
50. Build an RPG with JavaScript and HTML Canvas (Continued) - YouTube <https://www.youtube.com/watch?v=yTFIciZKG0M>
51. How to create page transition using vanilla javascript : r/webdev - Reddit https://www.reddit.com/r/webdev/comments/1gyi5rc/how_to_create_page_transition_using_vanilla/
52. On hydration: A non-technical perspective | bobae kang <https://bobaekang.com/blog/on-hydration-a-non-technical-perspective/>
53. What is Hydration? - YouTube <https://www.youtube.com/watch?v=R-BKadZWYnQ>
54. Why Efficient Hydration in JavaScript Frameworks is so Challenging https://www.reddit.com/r/javascript/comments/sjp2t1/why_efficient_hydration_in_javascript_frameworks/
55. Progressive Hydration - Patterns.dev <https://www.patterns.dev/react/progressive-hydration/>
56. Hydration is Pure Overhead - Builder.io <https://www.builder.io/blog/hydration-is-pure-overhead>
57. The Transition from Client-Side to Server-Side Rendering | CS 484 <https://www.cs.uic.edu/~ckanich/cs484/f24/readings/chapter-3-server-side-web-development/full-stack-react/index.html>
58. Rendering on the Web | Articles - web.dev <https://web.dev/articles/rendering-on-the-web>
59. What Is JavaScript Hydration For Single Page Applications? <https://www.debugbear.com/blog/javascript-hydration>
60. How do frameworks update the DOM without a virtual DOM? - SitePoint <https://www.sitepoint.com/community/t/how-do-frameworks-update-the-dom-without-a-virtual-dom/264399>
61. Hydration (web development) - Wikipedia [https://en.wikipedia.org/wiki/Hydration_\(web_development\)](https://en.wikipedia.org/wiki/Hydration_(web_development))
62. Using pushState and replaceState https://willschenk.com/labnotes/2024/using_push_state/
63. Mastering requestAnimationFrame: The Secret Weapon I ... <https://medium.com/@tharunbalaji110/mastering-requestanimationframe-the-secret-weapon-i-discovered-while-building-our-startups-c845cbc43a45>
64. Improve Web Performance With requestAnimationFrame <https://www.debugbear.com/blog/requestanimationframe>

65. Debugging Janky Scroll with Forced Reflow in React https://www.linkedin.com/posts/piyush-eon_%F0%9D%97%A7%F0%9D%97%B5%F0%9D%97%B6%F0%9D%98%80-%F0%9D%97%AF%F0%9D%97%BF%F0%9D%97%BC%F0%9D%98%84%F0%9D%98%80%F0%9D%97%B2%F0%9D%97%BF-%F0%9D%97%BF%F0%9D%97%B2%F0%9D%97%BB%F0%9D%97%B1%F0%9D%97%B2%F0%9D%97%BF%F0%9D%97%B6%F0%9D%97%BB%F0%9D%97%B4-activity-7417932779734196224-TCBo
66. The definitive guide to requestAnimationFrame() - Flavio Copes <https://flaviocopes.com/requestanimationframe/>
67. 60 FPS: Performant web animations for optimal UX <https://www.algolia.com/blog/engineering/60-fps-performant-web-animations-for-optimal-ux>
68. It gets a high-resolution timestamp, so you animate by ... <https://www.instagram.com/thefrontendround/reel/DcCzMXYAF3J/>
69. johnl28/rpg: The First Dungeon is an RPG game ... <https://github.com/johnl28/rpg>
70. Mastering the JavaScript History API: Dynamic Navigation ... <https://www.youtube.com/watch?v=1jsgvvXMtRY>
71. State · Design Patterns Revisited <https://gameprogrammingpatterns.com/state.html>
72. Article: Tame Your Game Code With State Machines https://www.reddit.com/r/gamedev/comments/45nn5i/article_tame_your_game_code_with_state_machines/
73. Classic Front End MVC with Vanilla Javascript <https://medium.com/@patrickackerman/classic-front-end-mvc-with-vanilla-javascript-7eee550bc702>
74. Is it ever a good idea to hardcode values into our ... <https://softwareengineering.stackexchange.com/questions/67982/is-it-ever-a-good-idea-to-hardcode-values-into-our-applications>
75. Vanilla JavaScript game tutorial from scratch : r/html5 - Reddit https://www.reddit.com/r/html5/comments/jxo47e/vanilla_javascript_game_tutorial_from_scratch/
76. Code a 2D Animation System That NEVER Breaks (Vanilla JavaScript) <https://www.youtube.com/watch?v=4L7A5Fy0CAE>
77. r/javascript - JCGE — A Vanilla JS 2D Game Engine, 5 Years in the Making https://www.reddit.com/r/javascript/comments/1rud0o6/jcge_a_vanilla_js_2d_game_engine_5_years_in_the/
78. Build a Reusable 2D Game Engine in JavaScript - YouTube <https://www.youtube.com/watch?v=4xx9ky3n8U8>
79. Building Async Page Transitions in Vanilla JavaScript - Codrops <https://tympanus.net/codrops/2026/02/26/building-async-page-transitions-in-vanilla-javascript/>

80. Vanilla JavaScript browser game with sprite sheets and collision detection https://www.reddit.com/r/learnjavascript/comments/kboggp/vanilla_javascript_browser_game_with_sprite/
81. Game Development with Vanilla JavaScript - YouTube https://www.youtube.com/playlist?list=PLYElE_rzEw_sowQGjRdvwh9eAEt62d_Eu
82. Build a Game with JavaScript and HTML Canvas [RPG Kit series] <https://www.youtube.com/watch?v=HmxNrlPx8iY>
83. Save current js object in history.pushState - javascript <https://stackoverflow.com/questions/35914413/save-current-js-object-in-history-pushstate>
84. 5.4 Session history and navigation — HTML5 <https://www.w3.org/TR/2011/WD-html5-20110113/history.html>
85. when to use pushState() versus replaceState() - Nick Colley <https://nickcolley.co.uk/2018/06/11/pushstate-vs-replacestate/>
86. Can you get the previous history state object in js? <https://stackoverflow.com/questions/36525412/can-you-get-the-previous-history-state-object-in-js>
87. Popstate and history API — the missing part - Medium <https://yedri.medium.com/popstate-and-history-api-the-missing-part-dc49f75676d>
88. PopStateEvent - Web APIs <https://mdn2.netlify.app/en-us/docs/web/api/popstateevent/>
89. We Can Finally Transition Display: None! | by Kenton de Jong | Medium <https://kentondejong.medium.com/we-can-finally-transition-display-none-cbb03831351f>
90. Transitioning Top-Layer Entries And The Display Property In CSS <https://www.smashingmagazine.com/2025/01/transitioning-top-layer-entries-display-property-css/>
91. Stuck in a wall... How do I store object methods in localStorage? - Reddit https://www.reddit.com/r/javascript/comments/6xk2tt/stuck_in_a_wall_how_do_i_store_object_methods_in/
92. window.history.pushState refreshing the browser <https://stackoverflow.com/questions/24307401/window-history-pushstate-refreshing-the-browser>
93. How to reload content on back and forward browser clicks <https://www.sitepoint.com/community/t/history-api-how-to-reload-content-on-back-and-forward-browser-clicks/303219>
94. Clarify history.pushState/replaceState behavior with ... <https://github.com/whatwg/html/issues/6207>
95. history.replaceState() does not refresh :target in Internet ... <https://community.spiceworks.com/t/history-replacestate-does-not-refresh-target-in-internet-explorer/535161>

96. pushState and popstate <https://www.youtube.com/watch?v=XRC0hOh4UGY>
97. pushState/onpopstate - go back multiple states <https://stackoverflow.com/questions/45327777/pushstate-onpopstate-go-back-multiple-states>
98. Virtual URL navigation using vanilla JavaScript <https://dev.to/pbouillon/virtual-url-navigation-using-vanilla-javascript-4o38>
99. JS library that has the the ability to serialize/re-hydrate ... <https://stackoverflow.com/questions/7839702/js-library-that-has-the-the-ability-to-serialize-re-hydrate-elements-state>
100. Functional HTML Rendering and State Management in Vanilla JS https://www.youtube.com/watch?v=yBg8w_5THS4
101. Persisting DOM State in Vanilla JS : r/learnjavascript https://www.reddit.com/r/learnjavascript/comments/101ep0q/persisting_dom_state_in_vanilla_js/
102. Mastering State Management in JavaScript Without a ... <https://python.plainenglish.io/mastering-state-management-in-javascript-without-a-framework-130935335301>
103. Build a state management system with vanilla JavaScript <https://css-tricks.com/build-a-state-management-system-with-vanilla-javascript/>
104. How to manage HTML DOM with vanilla JavaScript only? <https://news.ycombinator.com/item?id=22758218>
105. Vanilla JavaScript Patterns for Modern Browsers - Library <https://www.grizzlypeaksoftware.com/library/vanilla-javascript-patterns-for-modern-browsers-9jkhape>
106. How State Management works? Dead simple SM in Vanilla ... <https://dev.to/vijaypushkin/dead-simple-state-management-in-vanilla-javascript-24p0>
107. Debugging layout repaint issues triggered by CSS Transition <https://dzhavat.github.io/2021/02/18/debugging-layout-repaint-issues-triggered-by-css-transition.html>
108. Workarounds for CSS Transition Visibility to Fade and ... <https://taccgl.org/blog/css-transition-visibility.html>
109. The Color Lie 3D Browser-Based Platformer with Three.js ... https://www.linkedin.com/posts/muhammad-arman-saleem_aiseekho2026-vibekaregakistan-aiseekho-activity-7456026303696134145-8Dh6
110. Quick Tip — Too Much DOM Manipulations Can Hurt You <https://medium.com/@gilfink/quick-tip-too-much-dom-manipulations-can-hurt-you-b63cba5198a4>
111. How large DOM sizes affect interactivity, and what you can ... <https://web.dev/articles/dom-size-and-interactivity>

112. Performance tips for JavaScript Game Developers https://www.reddit.com/r/incremental_games/comments/mwx2xd/performance_tips_for_javascript_game_developers/
113. Performance Optimization in DOM Manipulation <https://blog.carlosrojas.dev/performance-optimization-in-dom-manipulation-6669ae153847>
114. DOM Performance (Reflow & Repaint) (Summary) <https://gist.github.com/faressoft/36cdd64faae21ed22948b458e6bf04d5>
115. An Overview of the JavaScript History API <https://www.sitepoint.com/an-overview-and-usage-of-javascript-history-api/>
116. Four new CSS features for smooth entry and exit animations | Blog <https://developer.chrome.com/blog/entry-exit-animations>
117. Using sessionStorage in a javascript application | by jck <https://levelup.gitconnected.com/using-sessionstorage-in-a-javascript-application-9c920a546465>
118. State Management in Vanilla JS: 2026 Trends <https://www.vibidsoft.com/blog/state-management-in-vanilla-js-2026-trends/>
119. Implementing Simple SPA Routing Using Vanilla JavaScript https://dev.to/am_pra_veen/implementing-simple-spa-routing-using-vanilla-javascript-25b6
120. Simple state management with vanilla JavaScript | by Vadim Valuev <https://medium.com/@novosibcool/simple-state-management-with-vanilla-javascript-66fcd79380d4>
121. Ask HN: What happened to vanilla HTML/CSS/JS development? <https://news.ycombinator.com/item?id=32780191>
122. Data reactivity with vanilla JS | Go Make Things <https://gomakethings.com/articles/data-reactivity-with-vanilla-js/>
123. Vanilla JavaScript and HTML - No frameworks. No libraries. No problem. <https://dev.to/pluralsight/vanilla-javascript-and-html-no-frameworks-no-libraries-no-problem-2n99?ref=johnpapa.net>
124. Mastering the History API in JavaScript: Modern Navigation ... <https://rahulsoni0809.medium.com/mastering-the-history-api-in-javascript-modern-navigation-techniques-for-spas-5c06ac1e8b64>
125. How to update the browser URL without refreshing ... <https://gomakethings.com/articles/how-to-update-the-browser-url-without-refreshing-the-page-using-the-vanilla-js-history-api/>
126. Manipuler l'historique du navigateur - Les API Web | MDN https://developer.mozilla.org/fr/docs/Web/API/History_API

127. Make anchor link go some pixels above where it's linked to <https://stackoverflow.com/questions/17534661/make-anchor-link-go-some-pixels-above-where-its-linked-to>
128. Vertical Platformer Game Tutorial with JavaScript and HTML ... <https://www.youtube.com/watch?v=rTVoyWu8r6g>
129. HTML & CSS Project for Beginners (2024) <https://www.youtube.com/watch?v=lwpCppqHCXs>
130. Linking to a specific section of another page is NOT ... <https://forum.freecodecamp.org/t/linking-to-a-specific-section-of-another-page-is-not-working-html-going-nuts/264773>
131. Coding CSS, HTML and Javascript - Customize with code <https://forum.squarespace.com/topic/226549-coding-css-html-and-javascript/>
132. Top Down Game Camera and Movement in HTML, CSS ... <https://www.youtube.com/watch?v=H3Fn33lYuE0>
133. Making an interactive text adventure game with JavaScript ... <https://medium.com/@dimterion/making-an-interactive-text-adventure-game-with-javascript-html-and-css-298b35c8ea96>
134. Build a Pixel Art Maker with HTML, CSS, and JavaScript <https://www.codedex.io/projects/build-a-pixel-art-maker-with-html-css-and-javascript>
135. A simple JavaScript that lets you use your browser's back/forward ... - Gist <https://gist.github.com/3186244>
136. Push State / Pop State example - GitHub Gist <https://gist.github.com/3953286>
137. Using window.history.pushState to add state other than null to browser ... <https://github.com/vercel/next.js/discussions/62930>
138. Popstate on page's load in Chrome - Stack Overflow <https://stackoverflow.com/questions/6421769/popstate-on-pages-load-in-chrome>
139. The artificial triggering of 'onpopstate' in pushState() causes losing of steps <https://github.com/browserstate/history.js/issues/47>
140. pushState javascript router to vanilla js framework - GitHub <https://gist.github.com/97b97d33e0412af60939084f125e7924>
141. Building Web Components with Vanilla JavaScript <https://dev.to/aspittel/building-web-components-with-vanilla-javascript-jho>